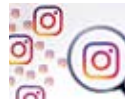




Noise's Smart Eyewear

Noise unveiled its first ever eyewear in India Noisei which is priced at ₹5,999. <https://dgit.in/jul22-65>



Instagram uses AI to know your age

Instagram is testing new methods for age verification using Artificial Intelligence. <https://dgit.in/jul22-66>

```
io.on("connection", (socket)
=> {
    socket.emit("my-mes-
sage", "Hello World!")
})
```

Now, if you have configured every-
thing right, you will see "Hello World!"
logged out on your browser's console.



Just make sure that whatever event
you "emit" from the server should be the
same event that you're listening for on the
client. In this case, it was "my-message"
but in reality, you can name it literally
anything you want.

It's as simple as that!

Completing The App

Now, we want communication to
happen between two or more different
hosts, right? Well, guess what - each
time you connect to a web socket from
a client, you are automatically allotted
a different and unique ID. This means
that the only work that you will have
to do is to emit or broadcast messages
from the server to the client and vice
versa and socket.io will do all the heavy
lifting for you. We made a dummy app
and printed out the ID in 4 different
instances simultaneously. You can see
that each ID is unique for each instance.

Client Side

On the client side, you will need to
take care of two things. First, when-
ever you press Send on the client side,
the text from the input field should be
emitted via a web socket. To do that,
we simply call a function whenever the
Send button is pressed :-

```
function sendMessage(text)
{
```

```
    socket.
emit("user-mes-
sage", text)
}
```

The text parameter
in this function is the
text from the input
field. The last thing to
take care of is when-

ever a client

receives a

message from the server side,
it should display it. To do
that, listen for the "server-
message" event on the client
side and the data we get from
it will be passed to another
function which will display
the text into its respective
container.

```
    socket.on("server-mes-
sage", (data) => {
    displayServerMessage(data)
    })
```

Server Side

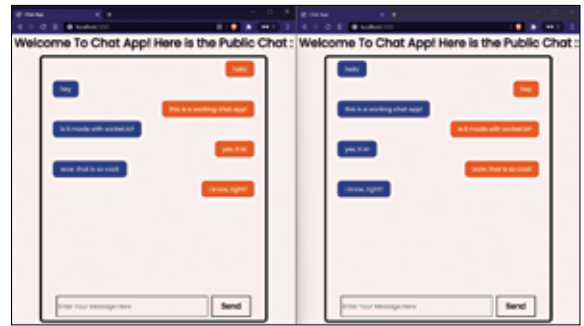
On the server side, things are pretty
simple. You just have to do one thing.
As soon as you're connected, you
need to listen for an incoming user
message. When that message arrives,
you need to broadcast it to all the
other connected sockets.

```
io.on("connection", (socket)
=> {
    socket.on("user-mes-
sage", (data) => {
        socket.broad-
cast.emit("server-message",
data)
    })
})
```

And that's pretty much all there is
to it. That is LITERALLY all the code
needed to run a real time chat app. On
the top right of the page is a screen-
shot of a working demo of the app.

Broadcast vs Emit

Earlier in the article, we
talked about broadcast
and emit. The main
difference between the
two is that whenever
you "emit" an event,



that event will be caught by all the
different web sockets listening for it,
INCLUDING YOU. This means that
in this code :- `io.on("connection",
(socket) => {
 socket.on("user-mes-
sage", (data) => {
 socket.broad-
cast.emit("server-message",
data)
 })
})`

If we change CODE: `socket.
broadcast.emit("server-message",
data)` to CODE: `io.emit("server-mes-
sage", data)` then whenever we send
a message from the client it will be
received by the server and the client
too! It looks something like this
picture below.

However, if we write CODE: `socket.
broadcast.emit("server-message", data)`
instead of CODE: `io.emit("server-mes-
sage", data)` then that means that the
client will emit the event to everybody
but itself. This, in our case results in the
desired behaviour.

CONCLUSION

We hope you had a lot of fun learning
about web sockets and socket.io.

We definitely had a lot of fun while
building this project and if you decide
to make a chat app of your own, don't
forget to give us an exclusive look
__. The full source code to this
project can be found here :- [https://
dgit.in/KJGithub](https://dgit.in/KJGithub).

